```
# ============================
# COMPONENT I: LSTM MODEL
# ============================

import numpy as np
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense

# ----------------------------
# DATASET
# ----------------------------
data = """machine learning models learn patterns from data
sequence models process data step by step
recurrent neural networks are designed for sequential tasks
rnn models maintain hidden states across time steps
long short term memory networks solve long dependency problems
lstm uses gates to control information flow
gru models simplify the lstm architecture
sequence prediction is useful in many applications
language modeling predicts the next word in a sentence
speech recognition processes audio sequences
time series forecasting predicts future values
music generation creates new melodies
generative models learn probability distributions
they generate new samples similar to training data
sequence generation is widely used in artificial intelligence
deep learning improves sequence modeling performance"""

# ----------------------------
# TOKENIZATION
# ----------------------------
tokenizer = Tokenizer()
tokenizer.fit_on_texts([data])
total_words = len(tokenizer.word_index) + 1

# ----------------------------
# CREATE SEQUENCES
# ----------------------------
input_sequences = []

for line in data.split('\n'):
    token_list = tokenizer.texts_to_sequences([line])[0]
    for i in range(1, len(token_list)):
        input_sequences.append(token_list[:i+1])

max_len = max(len(x) for x in input_sequences)

input_sequences = pad_sequences(input_sequences, maxlen=max_len, padding='pre')

X = input_sequences[:, :-1]
y = input_sequences[:, -1]
y = tf.keras.utils.to_categorical(y, num_classes=total_words)

# ----------------------------
# MODEL
# ----------------------------
model = Sequential([
    Embedding(total_words, 64, input_length=max_len-1),
    LSTM(100),
    Dense(total_words, activation='softmax')
])

model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])

# ----------------------------
# TRAIN
# ----------------------------
model.fit(X, y, epochs=200, verbose=1)

# ----------------------------
# GENERATE TEXT
# ----------------------------
def generate_text(seed_text, next_words=5):
    for _ in range(next_words):
        token_list = tokenizer.texts_to_sequences([seed_text])[0]
        token_list = pad_sequences([token_list], maxlen=max_len-1, padding='pre')

        predicted = np.argmax(model.predict(token_list), axis=-1)
```

```
        predicted = np.argmax(model.predict(token_list), axis=-1)

        for word, index in tokenizer.word_index.items():
            if index == predicted:
                seed_text += " " + word
                break

    return seed_text

print(generate_text("machine learning", 5))# ============================
# COMPONENT I: LSTM MODEL
# ============================

import numpy as np
import tensorflow as tf
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense

# ------------------------------
# DATASET
# ------------------------------
data = """machine learning models learn patterns from data
sequence models process data step by step
recurrent neural networks are designed for sequential tasks
rnn models maintain hidden states across time steps
long short term memory networks solve long dependency problems
lstm uses gates to control information flow
gru models simplify the lstm architecture
sequence prediction is useful in many applications
language modeling predicts the next word in a sentence
speech recognition processes audio sequences
time series forecasting predicts future values
music generation creates new melodies
generative models learn probability distributions
they generate new samples similar to training data
sequence generation is widely used in artificial intelligence
deep learning improves sequence modeling performance"""

# ------------------------------
# TOKENIZATION
# ------------------------------
tokenizer = Tokenizer()
tokenizer.fit_on_texts([data])
total_words = len(tokenizer.word_index) + 1

# ------------------------------
# CREATE SEQUENCES
# ------------------------------
input_sequences = []

for line in data.split('\n'):
    token_list = tokenizer.texts_to_sequences([line])[0]
    for i in range(1, len(token_list)):
        input_sequences.append(token_list[:i+1])

max_len = max(len(x) for x in input_sequences)

input_sequences = pad_sequences(input_sequences, maxlen=max_len, padding='pre')

X = input_sequences[:, :-1]
y = input_sequences[:, -1]
y = tf.keras.utils.to_categorical(y, num_classes=total_words)

# ------------------------------
# MODEL
# ------------------------------
model = Sequential([
    Embedding(total_words, 64, input_length=max_len-1),
    LSTM(100),
    Dense(total_words, activation='softmax')
])

model.compile(loss='categorical_crossentropy', optimizer='adam', metrics=['accuracy'])

# ------------------------------
# TRAIN
# ------------------------------
model.fit(X, y, epochs=200, verbose=1)

# ------------------------------
# GENERATE TEXT
#
```

```
# ------------------------------
def generate_text(seed_text, next_words=5):
    for _ in range(next_words):
        token_list = tokenizer.texts_to_sequences([seed_text])[0]
        token_list = pad_sequences([token_list], maxlen=max_len-1, padding='pre')

        predicted = np.argmax(model.predict(token_list), axis=-1)

        for word, index in tokenizer.word_index.items():
            if index == predicted:
                seed_text += " " + word
                break

    return seed_text

print(generate_text("machine learning", 5))
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedding.py:100: UserWarning: Argument `input_le
  warnings.warn(
Epoch 1/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 5s 17ms/step - accuracy: 0.0000e+00 - loss: 4.4669
Epoch 2/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.0632 - loss: 4.4569
Epoch 3/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.0526 - loss: 4.4482
Epoch 4/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 23ms/step - accuracy: 0.0526 - loss: 4.4378
Epoch 5/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.0526 - loss: 4.4233
Epoch 6/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.0526 - loss: 4.4034
Epoch 7/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.0526 - loss: 4.3709
Epoch 8/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.0526 - loss: 4.3137
Epoch 9/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.0526 - loss: 4.2415
Epoch 10/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.0526 - loss: 4.2210
Epoch 11/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.0526 - loss: 4.2066
Epoch 12/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.0526 - loss: 4.1677
Epoch 13/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 17ms/step - accuracy: 0.0526 - loss: 4.1352
Epoch 14/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.0737 - loss: 4.1098
Epoch 15/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.0947 - loss: 4.0728
Epoch 16/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.0947 - loss: 4.0380
Epoch 17/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.1053 - loss: 3.9974
Epoch 18/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.1158 - loss: 3.9508
Epoch 19/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.1474 - loss: 3.9029
Epoch 20/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.1684 - loss: 3.8495
Epoch 21/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.1579 - loss: 3.7875
Epoch 22/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 19ms/step - accuracy: 0.1474 - loss: 3.7228
Epoch 23/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.1684 - loss: 3.6640
Epoch 24/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.1789 - loss: 3.5980
Epoch 25/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.1789 - loss: 3.5398
Epoch 26/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.1789 - loss: 3.4754
Epoch 27/200
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.1895 - loss: 3.4105
Epoch 28/200
```